

Haldia Institute Of Technology.



A Project Report for Inhouse Training

**In the partial fulfilment of the award of the degree
of B.Tech.**

AI-Powered Smart Energy Consumption Predictor

Submitted by: Team Volt Squad

Members:

· Ayushi Rani :

University Roll-10301623021

· Somya Ranjan:

University Roll-10300323154

· Aranya Roy:

University Roll-10300223033

Submitted to: Debasish Sahoo Sir

Abstract

Energy consumption forecasting plays a vital role in modern smart cities and sustainable development. This project, AI-Powered Smart Energy Consumption Predictor, focuses on predicting household energy usage using machine learning techniques. The system considers multiple real-world factors such as household size, temperature, air-conditioner availability, and peak-hour usage to estimate total energy consumption. The project integrates a web-based frontend with a Flask-powered backend and a trained machine learning regression model.

1. Introduction

1.1 Project Domain

The domain of this project is Smart Energy Management, which lies at the intersection of Artificial Intelligence, Machine Learning, and Sustainable Technology. Smart energy systems aim to optimize electricity consumption, reduce wastage, and support eco-friendly practices.

1.2 Motivation

With increasing global energy demands and climate change concerns, efficient energy utilization has become critical. Households often lack tools to understand or predict their energy usage. This project addresses this gap by providing an AI-driven prediction system that helps users anticipate consumption and make informed decisions.

1.3 Objectives

- To design a machine learning model that predicts household energy consumption
- To build an interactive web interface for user input
- To deploy a backend API using Flask
- To integrate frontend and backend seamlessly
- To analyze and evaluate prediction performance

2. Literature Review

Previous research in energy prediction has widely used regression models, neural networks, and ensemble learning techniques. Studies show that factors such as weather conditions, appliance usage, and occupancy significantly influence energy consumption. Traditional statistical models often fail to capture nonlinear relationships, making machine learning a preferred approach.

3. Dataset Description

3.1 Dataset Overview

The dataset used for this project consists of household-level energy consumption data. Each record represents a household with associated features affecting electricity usage.

3.2 Features Description

- Household Size: Number of people living in the house
- Average Temperature (°C): Environmental temperature affecting cooling/heating needs
- Has AC: Binary feature (1 = AC available, 0 = No AC)
- Peak Hour Usage (kWh): Electricity consumed during peak hours
- Target Variable: Total Energy Consumption (kWh)

3.3 Data Source

The dataset is either synthetically generated or collected from publicly available energy consumption datasets and preprocessed for training.

3.4 Data Preprocessing

- Handling missing values
- Feature scaling (if required)
- Train-test split
- Conversion to numerical format

4. System Architecture

4.1 Overall Architecture

The system follows a client-server architecture:

- Frontend: HTML, CSS, JavaScript
- Backend: Flask REST API
- Machine Learning Model: Trained using Python

4.2 Workflow

- User enters input values in the web interface
- Data is sent to the Flask backend via POST request
- Backend loads trained ML model
- Model predicts energy consumption
- Result is returned and displayed to the user

5. Technology Stack

5.1 Frontend Technologies

- HTML5
- CSS3 (Glassmorphism UI)
- JavaScript (Fetch API)

5.2 Backend Technologies

- Python
- Flask
- Flask-CORS

5.3 Machine Learning Libraries

- NumPy
- Scikit-learn
- Joblib

6. Machine Learning Model Description

6.1 Model Type

This project uses a Regression Model since the output variable (energy consumption) is continuous.

6.2 Algorithm Used

Linear Regression and Random Forest Regression

6.3 Why Regression?

Regression techniques are highly suitable for this project because the objective is to predict a continuous numerical value, namely household energy consumption measured in kilowatt-hours (kWh). Regression models are designed to establish a mathematical relationship between one dependent variable and multiple independent variables.

In this project, Linear Regression and Random Forest Regression are considered. Linear Regression models the relationship between input features and output using a linear equation of the form:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_4 + \epsilon$$

where Y represents the predicted energy consumption, X_1, X_2, X_3, X_4 represent input features such as household size, temperature, AC usage, and peak hour usage, β represents model coefficients, and ϵ denotes the error term. While Linear Regression provides simplicity and

interpretability, Random Forest Regression improves prediction accuracy by combining multiple decision trees and reducing overfitting. Random Forest captures non-linear relationships between variables, making it more effective when energy consumption patterns are complex. Therefore, regression models are ideal for identifying relationships between input parameters and predicting numerical energy consumption values.

6.4 Model Training

Model training begins with mapping selected input features to the target variable, total energy consumption. The dataset is split into training and testing sets to evaluate model performance on unseen data. For Linear Regression, the model learns optimal coefficient values by minimizing the Mean Squared Error (MSE), calculated as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where y_i represents actual energy values and $\hat{y}_i$ represents predicted values.

For Random Forest Regression, multiple decision trees are trained on random subsets of the dataset, and the final prediction is obtained by averaging predictions from all trees. This ensemble approach improves robustness and reduces variance.

Model performance is evaluated using Mean Squared Error (MSE) and R^2 score. The R^2 score measures how well the model explains the variability in energy consumption and is calculated as:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

A higher R^2 value indicates better predictive performance. These evaluation metrics ensure the reliability and accuracy of the trained regression models before deployment.

7. Implementation Details

7.1 Project Folder Structure (GitHub Deployment)

The project is organized into separate frontend and backend directories to ensure modularity and ease of deployment.

Backend Folder: energy_prediction_backend/

- app.py – Flask application handling API requests
- energy_prediction_model.pkl – Trained machine learning model
- household_energy_consumption.csv – Dataset used for training
- requirements.txt – Python dependencies for deployment
- .gitignore – Ignored files for version control
- README.md – Backend documentation

Frontend Folder: energy_prediction_frontend/

- index.html – User interface for input and output
- style.css – Styling and animations (glassmorphism UI)
- script.js – JavaScript logic for API communication

The project is hosted on GitHub with a clean folder structure, making it suitable for collaboration and cloud deployment.

7.2 Frontend Implementation

The frontend consists of an interactive form that collects user input and sends it to the backend. CSS animations and gradients are used to enhance user experience.

7.2 Backend Implementation

Flask handles API requests. The trained model is loaded using Joblib and predictions are returned in JSON format.

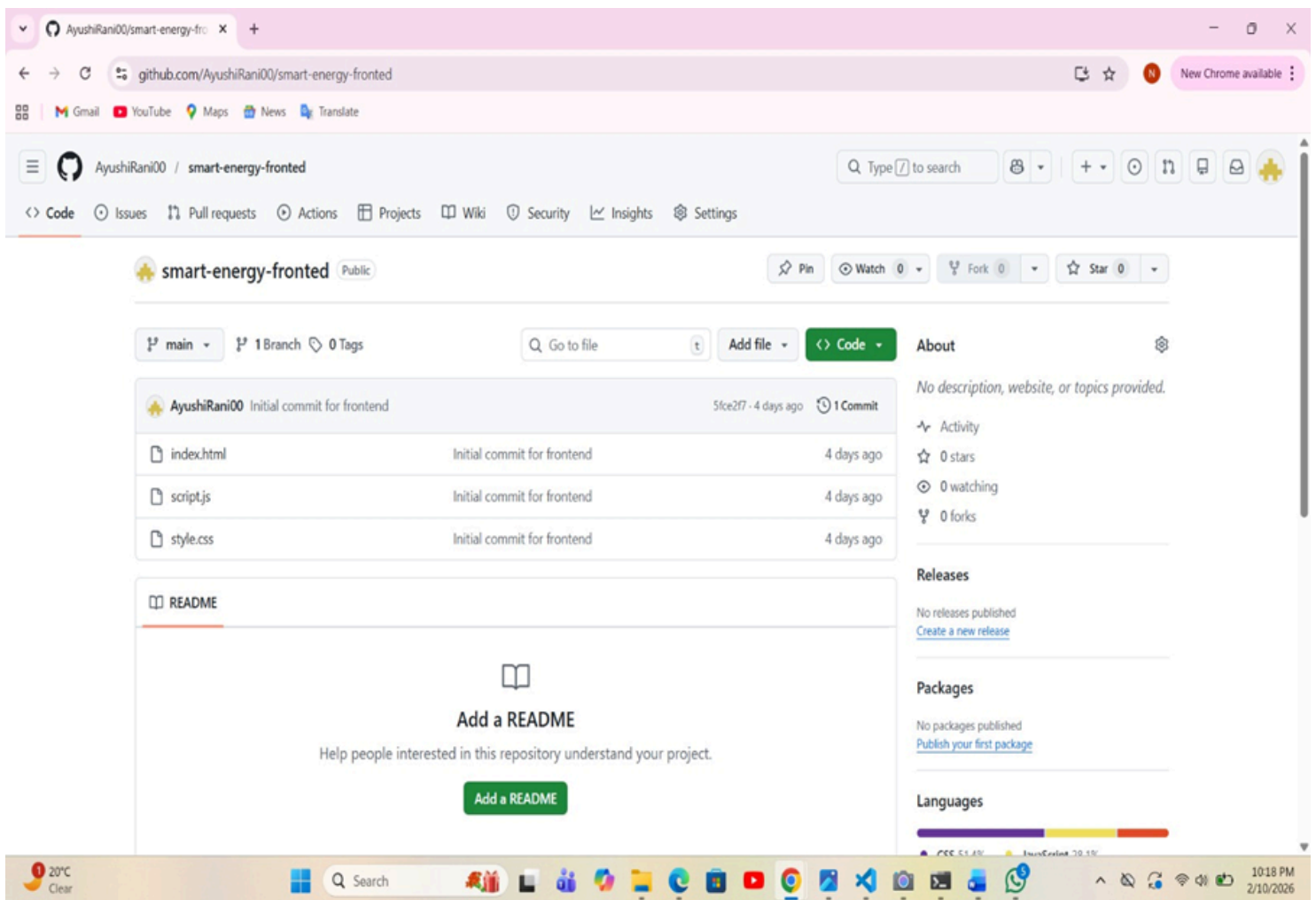
7.3 API Endpoint

POST /predict: Accepts JSON input and returns predicted energy consumption

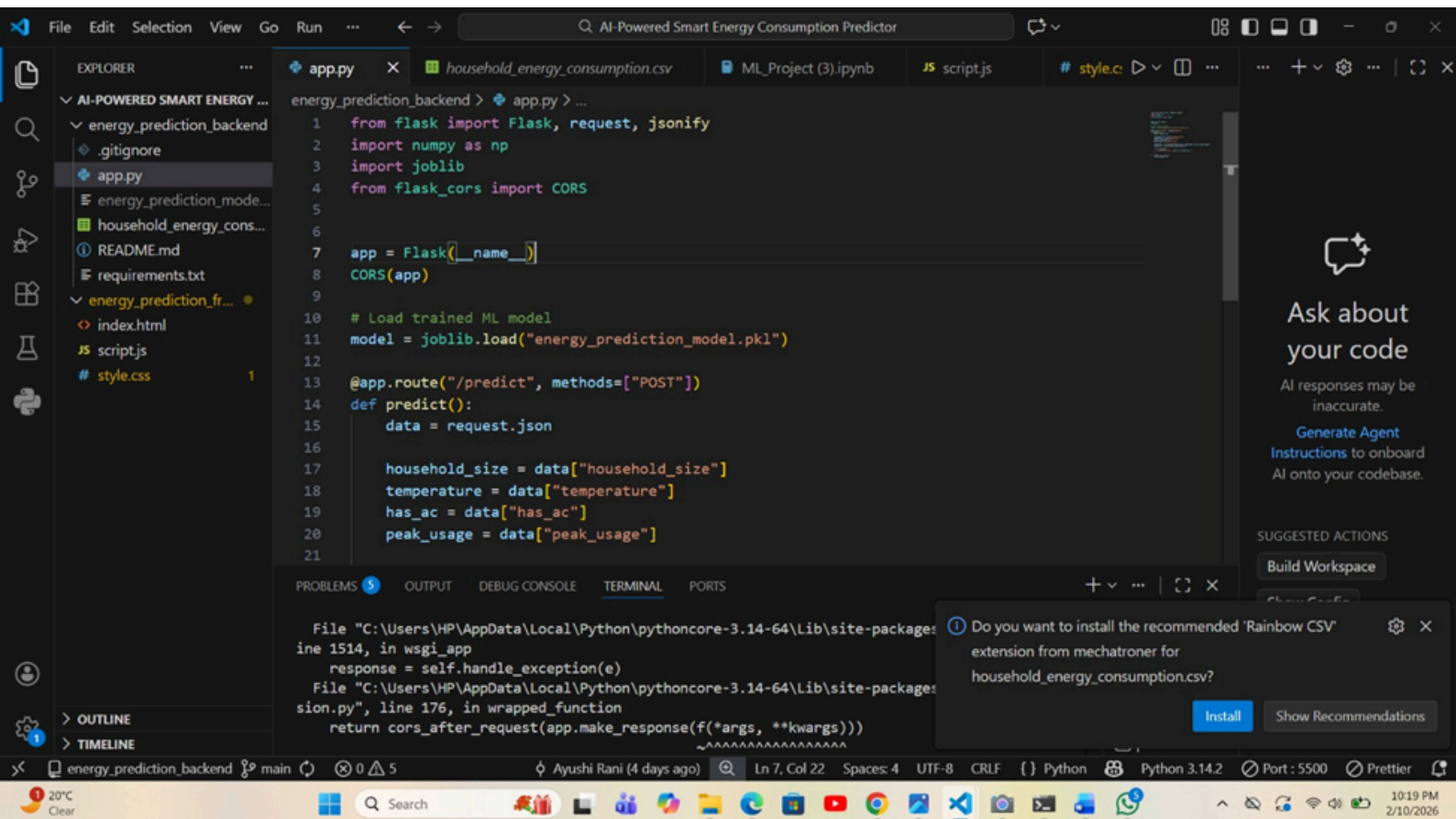
8. Screenshots

The following screenshots are included in the report to demonstrate implementation and deployment:

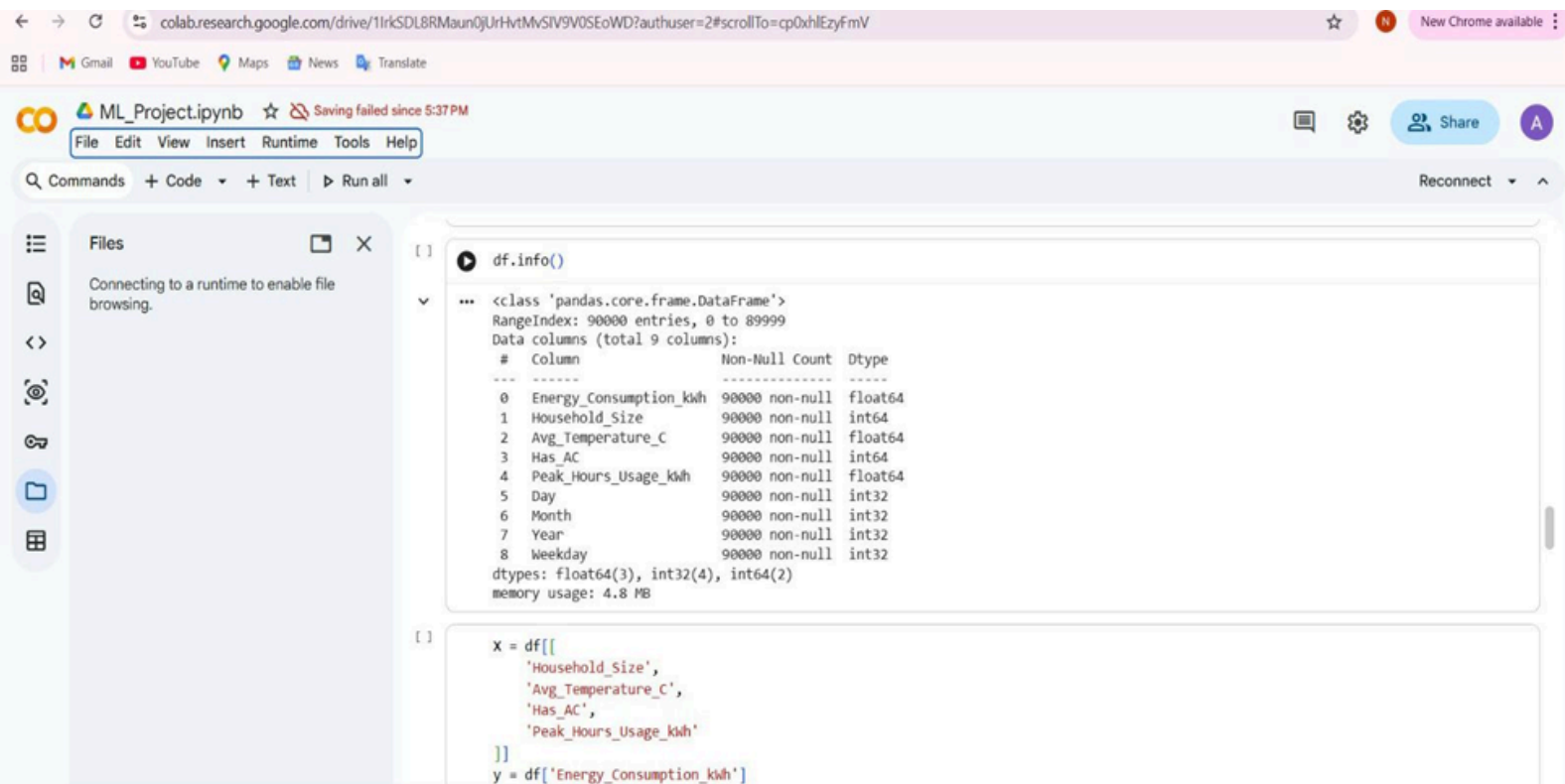
1. Screenshot of GitHub repository showing file and folder structure



3. Screenshot of Flask backend code (app.py)



4. Screenshot of machine learning model training code (Python/Jupyter Notebook)

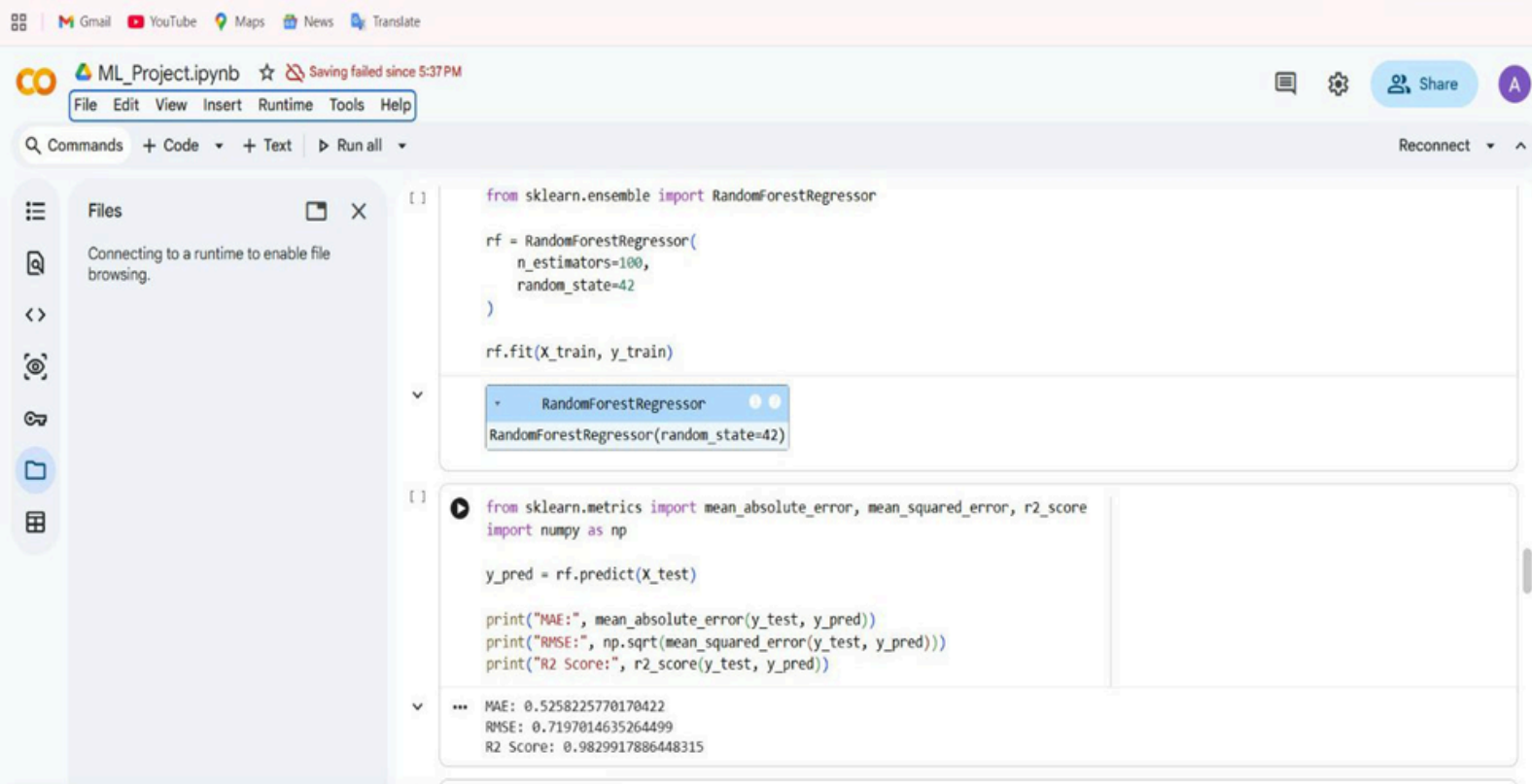


The screenshot shows a Google Colab notebook titled "ML_Project.ipynb". The interface includes a top navigation bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help" menus. A left sidebar contains icons for file management and a "Files" panel with the message "Connecting to a runtime to enable file browsing." The main area displays two code cells. The first cell contains the command `df.info()`, which has been executed, resulting in a detailed summary of the DataFrame. The second cell contains code for feature selection: `x = df[['Household_Size', 'Avg_Temperature_C', 'Has_AC', 'Peak_Hours_Usage_kwh']]` and `y = df['Energy_Consumption_kwh']`.

```
[ ] df.info()

... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 90000 entries, 0 to 89999
Data columns (total 9 columns):
 #   Column                      Non-Null Count  Dtype
---  -
 0   Energy_Consumption_kwh      90000 non-null float64
 1   Household_Size              90000 non-null int64
 2   Avg_Temperature_C           90000 non-null float64
 3   Has_AC                     90000 non-null int64
 4   Peak_Hours_Usage_kwh       90000 non-null float64
 5   Day                        90000 non-null int32
 6   Month                      90000 non-null int32
 7   Year                      90000 non-null int32
 8   Weekday                   90000 non-null int32
dtypes: float64(3), int32(4), int64(2)
memory usage: 4.8 MB

[ ] x = df[['Household_Size',
            'Avg_Temperature_C',
            'Has_AC',
            'Peak_Hours_Usage_kwh']]
y = df['Energy_Consumption_kwh']
```



The screenshot shows the same Google Colab notebook at a later stage. The first code cell now contains code to import `RandomForestRegressor` from `sklearn.ensemble`, create an instance with `n_estimators=100` and `random_state=42`, and fit it to the training data: `rf.fit(X_train, y_train)`. The second code cell imports metrics from `sklearn.metrics` and `numpy`, makes predictions on the test set, and prints the MAE, RMSE, and R2 Score. The output of the second cell shows the calculated metrics.

```
[ ] from sklearn.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

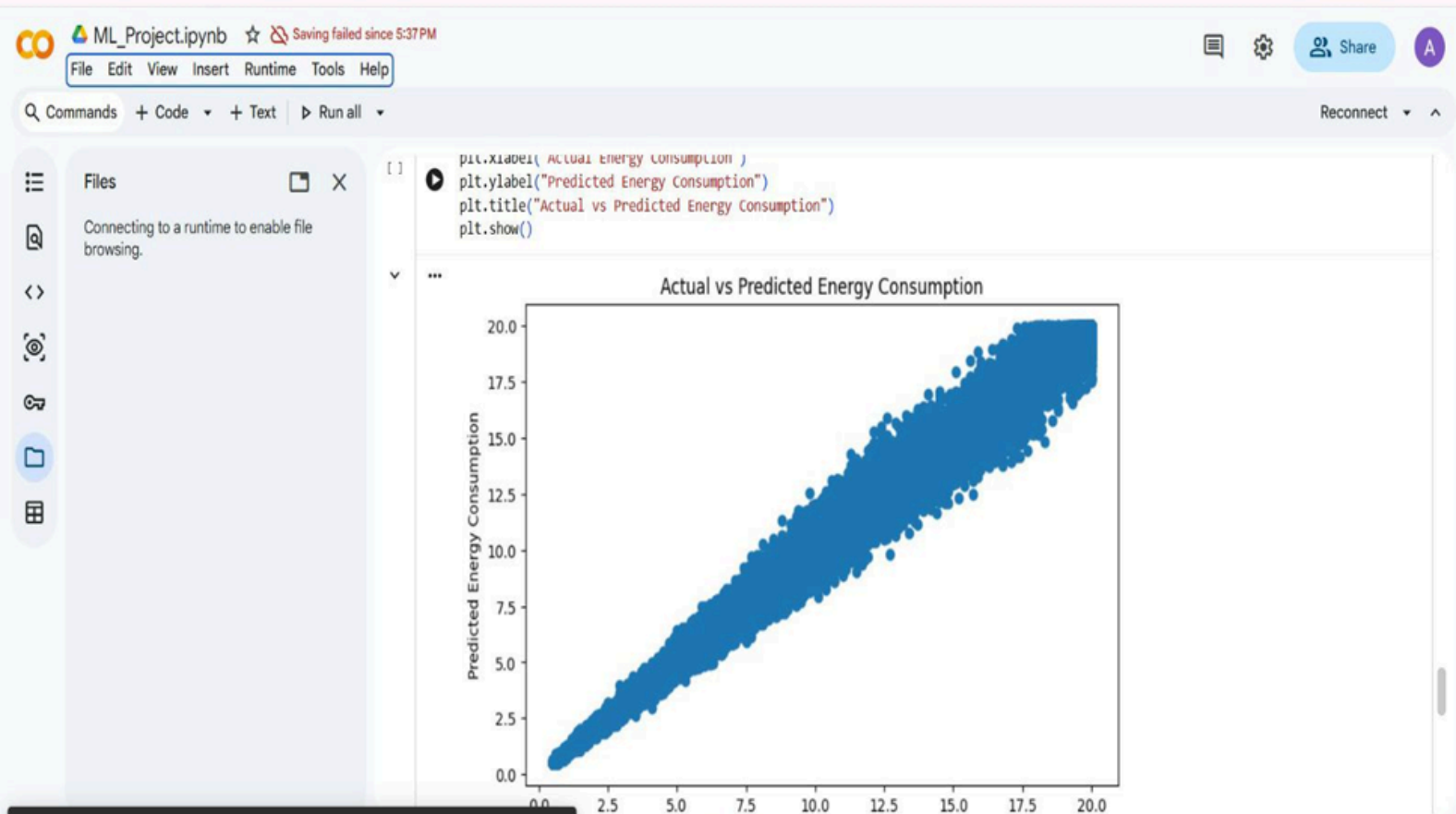
rf.fit(X_train, y_train)

[ ] from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

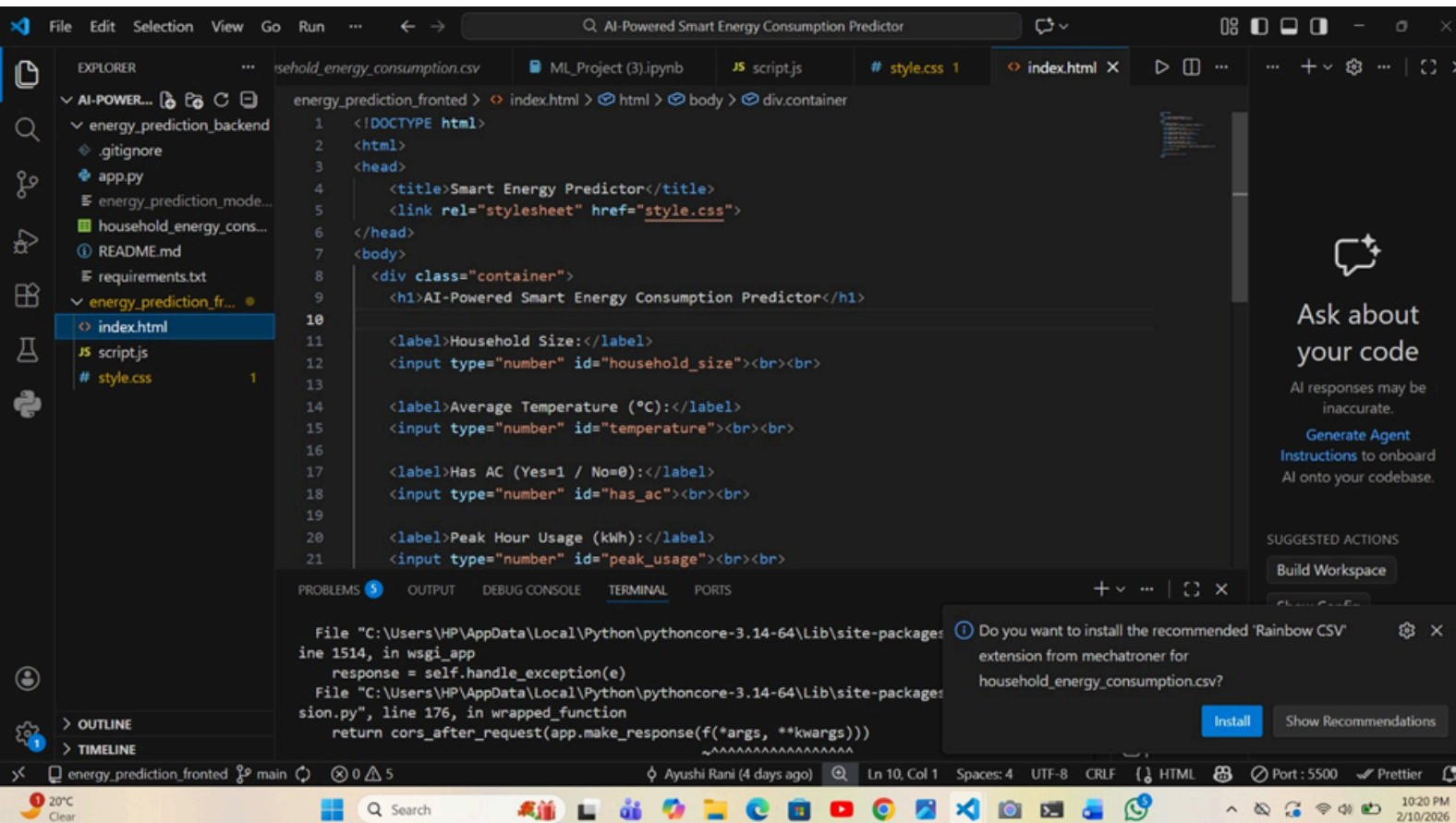
y_pred = rf.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2 Score:", r2_score(y_test, y_pred))

... MAE: 0.5258225770170422
RMSE: 0.7197014635264499
R2 Score: 0.9829917886448315
```

5. Screenshot of frontend UI (HTML/CSS)



9. Testing and Validation

9.1 Testing Approach

- Unit testing for backend API
- Manual testing for frontend UI

9.2 Results

The model produces realistic predictions based on input parameters, demonstrating good generalization on unseen data.

10. Deployment

10.1 GitHub Deployment

The complete project is deployed and version-controlled using GitHub. Separate folders for frontend and backend ensure clarity and maintainability. All required files including source code, dataset, trained model, and dependency list are pushed to the repository.

10.2 Backend Deployment

The Flask backend is deployed using a production-ready WSGI server (Gunicorn). Cross-Origin Resource Sharing (CORS) is enabled to allow communication between frontend and backend hosted on different platforms.

10.3 Frontend Deployment

The frontend is deployed as a static website and communicates with the backend using RESTful API calls. Environment-based backend URLs are used to support both local and cloud execution.

10.4 Local Deployment

Flask server running on localhost

10.2 Cloud Deployment

- Backend deployed on Render
- Frontend hosted using static hosting

11. Advantages of the System

- User-friendly interface
- Real-time predictions
- Scalable architecture
- Supports energy conservation

12. Limitations

- Limited dataset size
- Predictions depend on data quality
- Does not consider appliance-level consumption

13. Future Enhancements

- Integration with IoT smart meters
- Mobile application support
- Advanced deep learning models
- Real-time weather API integration

14. Final Observations and Results

The AI-Powered Smart Energy Consumption Predictor successfully demonstrates how machine learning can be applied to real-world energy management problems. The system accurately predicts energy consumption based on key influencing factors. This project highlights the importance of AI in building sustainable and intelligent systems. With further enhancements and real-world data integration, the model can be scaled for commercial and smart-city applications.

15. Conclusion

The AI-Powered Smart Energy Consumption Predictor project successfully demonstrates the use of machine learning techniques in the domain of smart energy management. The main goal of this project was to predict household energy consumption based on important influencing factors such as household size, temperature, air conditioner availability, and peak hour electricity usage. The system effectively meets this objective by providing accurate and reliable energy consumption predictions.

The project uses a regression-based machine learning model, which is suitable for predicting continuous numerical values.

Proper data preprocessing and feature selection helped improve the model's performance. The integration of the trained model with a Flask backend and a web-based frontend resulted in a complete end-to-end system that allows users to interact with the model easily.

The results obtained from the system reflect realistic energy consumption patterns. Observations show that factors such as household size, temperature, and air conditioner usage have a significant impact on electricity consumption. The project also provided practical experience in machine learning, backend development, frontend design, and API integration.

Although the system has certain limitations, such as dependency on dataset quality and lack of real-time data, it serves as a strong foundation for future improvements.

Overall, this project highlights the importance of artificial intelligence in energy efficiency and demonstrates how machine learning can be applied to solve real-world problems related to energy management.

References

1. Scikit-learn Documentation
2. Flask Official Documentation
3. Energy Consumption Datasets (Kaggle, UCI)
4. Research Papers on Smart Energy Systems